

Failure Tolerant Training With Persistent Memory Disaggregation Over CXL

Miryeong Kwon , Junhyeok Jang , Hanjin Choi, Sangwon Lee, and Myoungsoo Jung , KAIST and Panmnnesia, Daejeon, 34141, South Korea

This article proposes TRAININGCXL that can efficiently process large-scale recommendation datasets in the pool of disaggregated memory while making training fault tolerant with low overhead. To this end, we integrate persistent memory (PMEM) and graphics processing unit (GPU) into a cache-coherent domain as type 2. Enabling Compute Express Link (CXL) allows PMEM to be directly placed in GPU's memory hierarchy, such that GPU can access PMEM without software intervention. TRAININGCXL introduces computing and checkpointing logic near the CXL controller, thereby training data and managing persistency in an active manner. Considering PMEM's vulnerability, we utilize the unique characteristics of recommendation models and take the checkpointing overhead off the critical path of their training. Finally, TRAININGCXL employs an advanced checkpointing technique that relaxes the updating sequence of model parameters and embeddings across training batches. The evaluation shows that TRAININGCXL achieves $5.2\times$ training performance improvement and 76% energy savings, compared to the modern PMEM-based recommendation systems.

Deep learning-based recommendation systems take the majority of machine resources in diverse production servers and datacenters.¹ In practice, many production-level recommendation models (RMs) require highly accurate services to prevent hyperscalers from undesirable losses in revenues. This insists on large-sized models and feature vectors to train (i.e., embeddings), which are even much bigger than the largest deep neural-network-based model, such as transformers. For example, several studies have reported that the production-level RMs often consume more than tens of terabyte/peta-byte memory spaces.²

In addition, it is important for the RMs to be failure tolerant as they should be trained many days or weeks without accuracy degradation. To this end, the RMs periodically store their current training snapshots in persistent storage as checkpoints.³ Even though the

checkpoints are essential for system failure recovery, it is often considered the performance bottleneck in diverse computing domains, including the RMs.³

To address these challenges, several approaches have been proposed to employ high-performance solid-state drives (SSDs) and expand their host memory using SSDs as backend storage media.^{4,5} For example,⁴ store large-scale embedding tables while keeping only the feature vectors, frequently accessed from the host computing resources [central processing unit/graphics processing unit (CPU/GPU)], in their local memory. While this SSD-integrated memory expansion technique can handle the large-sized input data, they unfortunately suffer from severe performance degradation. This is because RM's embedding lookup tasks often generate small-sized reads with a random pattern whereas SSDs are optimized for other types of bulk input/output (I/O) operations.

Furthermore, all the prior approaches require explicit checkpoints for fault recovery. Since embedding updates on SSDs can degrade the training performance significantly, the existing RMs utilize the SSDs for only memory expansion purposes. Note that the write latency of SSDs is longer than the latency of all

0272-1732 © 2023 IEEE

Digital Object Identifier 10.1109/MM.2023.3237548

Date of publication 17 January 2023; date of current version 13 March 2023.

conventional memory operations by many orders of magnitude. The writes also introduce many internal tasks, such as garbage collection, making the training performance unacceptable in many cases.⁶

We propose TRAININGCXL that can efficiently process large-scale RM datasets in the underlying memory pool, disaggregated over Compute Express Link (CXL). TRAININGCXL makes deep learning training fault tolerant without imposing the checkpointing overhead as well. Our contributions can be summarized as follows.

WE PROPOSE TRAININGCXL THAT CAN EFFICIENTLY PROCESS LARGE-SCALE RM DATASETS IN THE UNDERLYING MEMORY POOL, DISAGGREGATED OVER COMPUTE EXPRESS LINK (CXL).

Intelligent CXL memory expansion: TRAININGCXL forms a nonvolatile memory expander (CXL-MEM) having many persistent memory modules (PMEM) as a type-2 of CXL 3.0.⁷ We also introduce type-2's device coherent (DCOH) agent to GPU, called CXL-GPU. Since TRAININGCXL integrates CXL-MEM and CXL-GPU into the same cache coherent domain, all the input/output embeddings are exchanged between those two without any software intervention running on the host CPU. In our architecture, CXL-MEM employs simple computing and checkpointing logic along with a type-2 endpoint controller, which can perform embedding operations and failure tolerance management near PMEM.

Batch-aware checkpoint: To achieve high training bandwidth, TRAININGCXL lets CXL-GPU and CXL-MEM perform multilayer perceptrons (MLPs) and embedding operations simultaneously. However, the model and embedding updates for the end of each batch process make the training latency yet longer. To address this, we propose batch-aware checkpoint that is aware of each shape of the individual batch and performs undo logging in background. Practically, such a background undo logging scheme is infeasible for most applications as the target location where the system needs to update is unavailable before their computation completes. Since we can know the next batch's embedding structure at each training stage in advance, our scheme logs appropriate models and embeddings to CXL-MEM in parallel with RM's training tasks.

Embedding lookup and checkpoint relaxation: While our batch-aware checkpoint can hide the

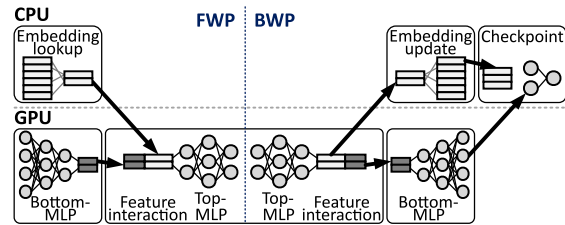


FIGURE 1. Training process of DLRM.

relatively long latency of CXL-MEM's writes, the training performance can yet degrade owing to read-after-write (RAW) caused by embedding operations between two adjacent batches; RMs read some of newly updated embeddings at the beginning of each batch, but the reads can be stalled because of the writes (for the embedding updates) issued right before. Unfortunately, the training also suffers performance degradation when our batch-aware checkpoint is slower than the MLP computation in CXL-GPU. TRAININGCXL relaxes the order of embedding lookups and checkpoints, and it reschedules them to remove the RAW conflict issues and checkpointing overhead. This relaxation can make CXL-GPU and CXM-MEM operations fully overlap each other, thereby improving the training bandwidth further.

Our evaluation results show that training performance can be improved by 5.2×. TRAININGCXL also achieves 76% energy savings, compared to modern PMEM-based recommendation systems.

BACKGROUND

RM Training

Meta AI's deep learning recommendation model (DLRM)⁸ is a representative RM used for personalized item recommendations. To achieve a higher accuracy and better representation capacity, DLRM exploits both sparse features (categorical information) and dense features (continuous information). Because of these distinct characteristics between the sparse and dense features, they are encoded by different types of processing operations before going through the main training task, called top-MLP. While the dense features are learned and encoded into input(s) of the top-MLP using conventional matrix-multiplication operations, called bottom-MLP, the sparse features are processed by a set of embedding operations (e.g., table lookup/update).

Figure 1 shows a single-batch training process of DLRM. Considering different levels of the computing intensiveness, the bottom-MLP operations are all performed in GPU, whereas the embedding operations are

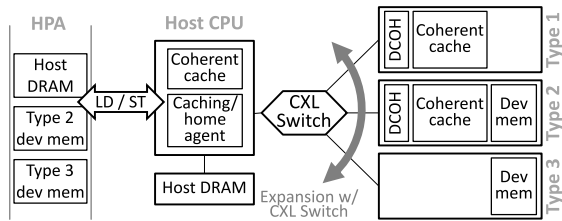


FIGURE 2. CXL architecture and CXL device types.

practically processed at the host side (CPU). For these embedding operations, the host reads the target embedding vectors from the underlying storage by referring to the corresponding table indices (residing in the sparse features). Note that production-scale embedding tables often exceed tens to hundreds terabytes, which unfortunately cannot be accommodated in the target system's local memory. The host then aggregates the retrieved embedding vectors using simple arithmetic (e.g., add/subtract) and generates new embedding vectors as another input of the top-MLP. As the bottom-MLP and embedding operations are simultaneously processed in different places (i.e., GPU and CPU), each encoded input data for the top-MLP can be prepared in parallel thereby saving the training time at some extent.⁴ To put the encoded inputs into a same vector space, GPU performs a feature interaction (e.g., concatenation) and the top-MLP, which trains the result of the feature interaction in this forward-propagation step (FWP). The backward-propagation (BWP) step of RM training is similar to FWP, but all its operations are processed in the reverse order. BWP takes gradient errors as an input (instead of FWP's input features) and updates model parameters/embeddings to improve the model accuracy. The gradient errors are calculated by the differences between the FWP results and labels (truth grounds). Note that all the updated model parameters (e.g., MLPs' weights and embeddings) should be checkpointed in the underlying storage for each end of all the batches.

Compute Express Link

CXL is an open industry standard interconnect, which allows multiple heterogeneous (computing) devices to share large-scale memory spaces in a cache-coherent manner. Figure 2 shows a system architecture that enables CXL. The system consists of three essential hardware components as follows: 1) CXL-enabled host CPU(s); 2) CXL switch(es); and 3) CXL device(s). CXL devices can be incarnated by leveraging the design of conventional peripheral devices (e.g., accelerator and

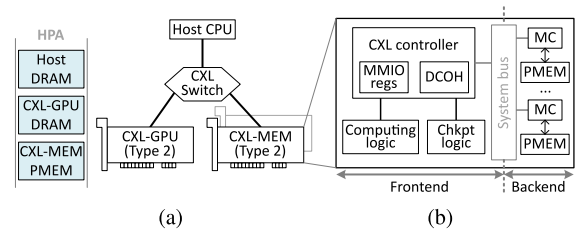


FIGURE 3. TRAININGCXL's system architecture. (a) Cache-coherent domain. (b) Internals of CXL-MEM.

memory expander), but they should accommodate appropriate CXL protocol interfaces for their specific purposes. CXL switches can interconnect the host CPU(s) and multiple CXL devices in a disaggregated manner. Note that a CXL network (per CPU's root complex) can have up to 4,095 CXL devices, which can all the host to secure sufficient memory space to use.⁷ All these CXL hardware components can have a unified memory space, referred to as host physical address (HPA) as their shared memory pool. To this end, CXL supports three subprotocols (CXL.io, CXL.cache, and CXL.mem).

Based on how to combine these subprotocols, CXL devices can be classified into type-1, type-2, and type-3. CXL.io is mandatory for all the types of hardware, allowing a CXL device to expose device registers to HPA as memory-mapped I/O (MMIO) registers. Using this protocol, the host CPU can discover or configure the underlying CXL devices (by reading/writing the MMIO registers). In addition to CXL.io, type-1 and type-3 need to support CXL.cache and CXL.mem, respectively. Note that type-2 is recommended to have all three subprotocols as it is designed toward having both computing and memory resources at the backend. The memory resources of type-2 is accessed by host through CXL.mem, while type-2's computing resources can buffer data residing in the HPA into its internal cache with CXL.cache. Internal cache's cacheline states are tracked by DCOH engine⁷ to guarantee cache coherency with other CXL devices.

PMEM DISAGGREGATION

TRAININGCXL disaggregates memory devices from CPU/GPU and integrates all of them into a single system over CXL. Our CXL-enabled memory expander employs PMEMs, which exhibit similar performance to DRAM and provide large-scale memory capacity. Thanks to the backend PMEM, TRAININGCXL can reduce the overhead imposed by heavy reads/writes (for embeddings), compared to the existing RM system. In addition, TRAININGCXL uses PMEM for memory

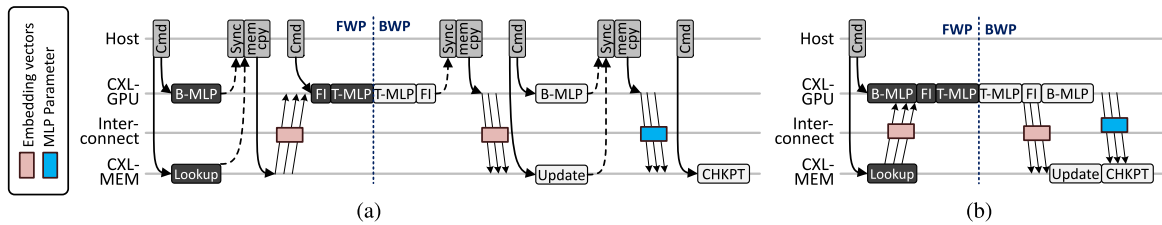


FIGURE 4. Comparison of RM training procedure. (a) Conventional software framework. (b) TrainingCXL's hardware.

expansion as well as leverages its nonvolatile characteristics to support failure tolerant training with low overhead. Finally, TRAININGCXL alleviates the data movement overhead between GPU and memory expander by exploiting the advantages that CXL sub-protocols offer.

System Architecture

Figure 3(a) shows an overview of the proposed TRAININGCXL's system architecture. TRAININGCXL proposes two CXL devices: a CXL-enabled GPU (CXL-GPU) and a PMEM-based memory expander (CXL-MEM). These are designed as type-2 and connected to a host CPU through CXL Switch(es). Because of their device type, CXL-MEM's internal memory can be exposed to CXL-GPU's local memory and vice versa. To accelerate RM processing, CXL-MEM manages all the embedding operations instead of the host CPU. Since the host CPU is free from processing embeddings in TRAININGCXL, it is only responsible for running RM training software (e.g., PyTorch or TensorFlow).

Designing CXL-MEM: Figure 3(b) illustrates CXL-MEM composed by the frontend and backend modules. The backend employs multiple PMEMs and corresponding memory controllers to achieve a high degree of data parallelism for large-scale embedding tables as well as model checkpoints. The aggregated memory space of PMEMs is exposed to CXL-MEM's frontend through system interconnect, which consists of: 1) a CXL controller that implements all the three CXL subprotocols; 2) a computing logic that processes embedding operations (lookup/update); and 3) a checkpointing logic that automatically creates the model checkpoints. To initialize computing logic, the host CPU sets CXL-MEM's MMIO registers with embedding vector length and learning rate, which are required by embedding operations. In addition, it is important to store the MLP parameters' memory address and the size of MLP parameters to allow checkpointing logic read MLP parameters from CXL-GPU. We will explain how this information is used by CXL-MEM's computing

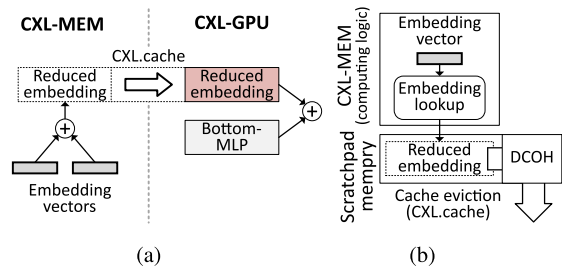


FIGURE 5. CXL-based automatic data movement. (a) Data placement. (b) Cache flush with DCOH.

and checkpointing logic with more details, in the "CXL-MEM's Checkpoint Support" section.

CXL-Based Automatic Data Movement

Since TRAININGCXL adopts a heterogeneous computing system, it is inevitable to move RM training's intermediate data across a set of CXL devices. However, this data movement has been managed by the RM training software running on the host CPU. To remove nonnegligible software overhead, TRAININGCXL proposes a hardware automation approach that moves data by leveraging CXL hardware components.

Performance impacts: Figure 4(a) demonstrates the performance impact of the conventional software-based data movement approaches. Using the FWP as an example, the software offloads bottom-MLP and embedding lookup to CXL-GPU and CXL-MEM, respectively. They notify the completion of the operation to the software through `cudaStreamSynchronize`; after that, the software can start to copy the new embedding vectors from CXP-MEM to CXL-GPU (e.g., `cudaMemcpy`) and request CXL-GPU to perform feature interaction and top-MLP. These software overhead can be eliminated with CXL-based automatic data movement, as shown in Figure 4(b). TRAININGCXL's CXL hardware components are responsible for data movement and RM training operations can be synchronized by checking whether all input data are ready or not.

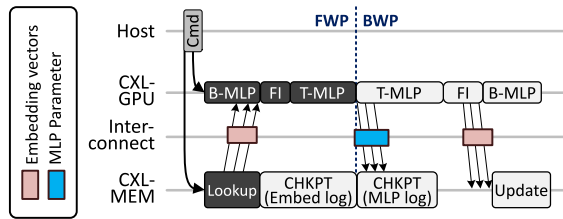


FIGURE 6. Execution of batch-aware checkpoint.

Design of hardware automation approach: The insight behind CXL-based automatic data movement is to leverage CXL.cache to transfer data. To move the data using CXL.cache, the data should be stored where it will be used as an operation input. For example, the input data of feature interaction (e.g., reduced embedding vector) should be stored in the CXL-GPU's device memory and cached in the CXL-MEM's internal cache, as shown in Figure 5(a). Then the CXL-MEM's DCOH flushes every cacheline of the reduced embedding vector, which is updated by embedding lookup, as shown in Figure 5(b). Similarly, in the case of BWP, the input data of the embedding update (e.g., embedding gradient) should be stored in the CXL-MEM's device memory and cached in the CXL-GPU. Then the embedding gradient is flushed by the CXL-GPU's DCOH and transferred to the CXL-MEM.

FAILURE TOLERANCE MANAGEMENT

This section will explain: 1) how the checkpointing overhead is removed from the critical path of RM training; and 2) how CXL-MEM's checkpointing logic can perform checkpointing automatically in the background.

Batch-Aware Checkpoint

The conventional SSD-based failure tolerance management is performed in a redo log manner. In other words, the updated embedding vectors and bottom/top-MLP parameters have been permanently stored at the end of each training epoch (before starting the next batch training). To take checkpointing off the critical path of RM training, TRAININGCXL proposes a batch-aware checkpoint that performs checkpointing in an undo log manner. It leverages RM training's characteristic that embedding vector indices to be updated can be known in advance even if the batch training is not completed yet. Since the sparse features include that information, RM training software sets them in the MMIO register for every batch to store embedding and MLP logs by utilizing the idle time of CXL-MEM, as shown in Figure 6.

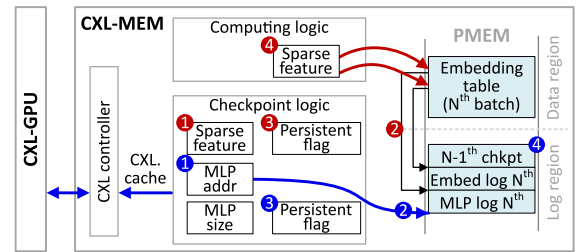


FIGURE 7. CXL-MEM's automatic checkpoint.

CXL-MEM's Checkpoint Support

To support batch-aware checkpoint, we first split the CXL-MEM's memory space into data and log regions. Each of these regions is for computing logic and checkpointing logic to store embedding tables and embedding/MLP logs, respectively. Since embedding/MLP logs are managed in the separated region, CXL-MEM can easily recover the RM model by referring to the log region when there is a power failure.

Embedding logging: Figure 7 shows how the checkpointing logic can generate checkpoint for embedding table. The checkpointing logic refers to the embedding vector indices from the sparse feature (①), and creates embedding log by copying the embedding vectors from the data region to the log region (②). When embedding log is permanently stored, checkpointing logic sets persistent flag as true (③). Since checkpointing logic already backed up the embedding vectors in the background, the embedding table in the data region can be directly updated during embedding update operation (④). Even if a power failure occurs during an embedding update, training can be resumed from that batch if the persistent flag is set.

MLP logging: Unlike embedding logging, the bottom/top-MLP parameters are stored in the CXL-GPU. Therefore, the checkpointing logic should send CXL.cache requests by referring to MLP parameters' memory address and size stored in the MMIO registers (①). When the MLP parameters are transferred from CXL-GPU, CXL-MEM stores them in the log region (②). By comparing the number of transferred MLP parameters and the MLP parameters' size stored in the MMIO register, checkpointing logic can recognize whether the MLP log is fully checkpointed. When all MLP parameters are safely stored, checkpointing logic sets the persistent flag as true for the MLP log (③). If the persistent flag is set for both the embedding and MLP log, checkpointing logic deletes the old checkpoint written in the previous batch (④).

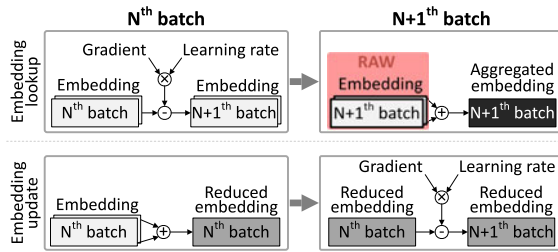


FIGURE 8. Dependency (top) versus relaxed (bottom).

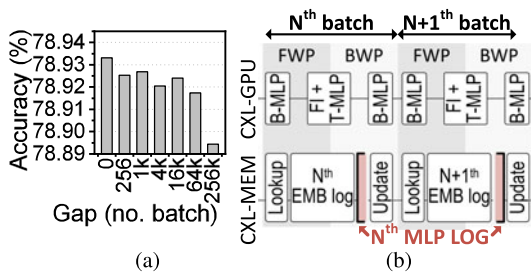


FIGURE 9. Relaxed batch-aware checkpoint. (a) Accuracy. (b) Schedule.

RELAXATION OF FAILURE TOLERANT TRAINING

TRAININGCXL proposes training relaxation techniques that can eliminate performance degradation caused by PMEM-based CXL-MEM design.

Relaxed Embedding Lookup

PMEM's read performance is similar to that of DRAM's. However, it can be degraded if a read is requested right after a write for the same physical layout of PMEM. This phenomenon is known as RAW,⁹ and can be found between the embedding update of the N^{th} batch and the embedding lookup of the $(N+1)^{th}$ batch. Kwon and Rhu¹⁰ analyzed that 80% of embedding vectors are trained for the consecutive batches, and this report indicates that RAW can be frequently observed during RM training.

Relaxation: As shown in the top of Figure 8, $(N+1)^{th}$ batch's embedding lookup has been performed when $(N+1)^{th}$ batch's embedding table is prepared (by completing the N^{th} batch). In other words, there is operation dependency between the N^{th} batch's embedding update and the $(N+1)^{th}$ batch's embedding lookup. However, since the embedding lookup and update are composed of addition/subtraction-based arithmetic, the operation dependency between embedding lookup and update can be relaxed by utilizing the commutative property

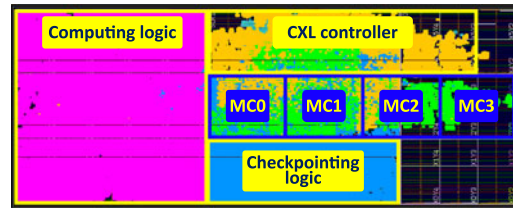


FIGURE 10. Prototype of CXL-MEM.

of addition. Thus, TRAININGCXL proposes relaxed embedding lookup, as shown in the bottom of Figure 8. The conventional $(N+1)^{th}$ batch's embedding lookup is now split into two steps. First, the embedding lookup is performed at N^{th} batch with the N^{th} batch's embedding table. After that, the embedding update for the reduced embedding vector is performed when the embedding gradient is ready.

Relaxed Batch-Aware Checkpoint

Although batch-aware checkpoint can mitigate the slow PMEM writes by overlapping checkpointing with CXL-GPU's feature interaction and top-MLP, checkpointing overhead can be observed if the checkpoint time is longer than the idle time that CXL-MEM can exploit.

Relaxation: Figure 9(a) shows the training accuracy according to the batch number difference between the embedding and the MLP log. As shown in this figure, the accuracy degradation satisfies the business needs (0.01%)³ even when the batch gap of the two logs differs by hundreds. This observation indicates that the bottom/top-MLP does not need to be checkpointed for every batch, although the embedding log should be permanently stored for every batch since the original embedding tables are updated for every batch. Thus, TRAININGCXL proposes a relaxed batch-aware checkpoint that can schedule MLP logging across multiple batches, as shown in Figure 9(b). Since the purpose of checkpointing relaxation is to hide its overhead from user experience, the MLP logging should be stopped when CXL-GPU completes the top-MLP operation. However, it is difficult for CXL-MEM to know whether CXL-GPU completes the MLP operation or not. Thus, CXL-GPU supports this checkpointing relaxation by responding to the CXL-MEM's CXL.cache request only when it processes feature interaction and top-MLP.

EVALUATION

Hardware prototype: The CXL-enabled RISC-V host CPU, CXL switch, CXL-GPU, and CXL-MEM are prototyped using a set of Xilinx Alveo U200s and a customized FPGA hardware devices. Figure 10 shows CXL-MEM's hardware prototype as an example, which

TABLE 1. Test environment.

	Embedding table storage	CPU	Memory	GPU
SSD	Intel 750 800 GB	Intel i5- 9600 K 3.7 GHz	4×16 GB DRAM	NVIDIA RTX 3090 (Emulated by Vortex)
PMEM	Intel Optane PMEM 512 GB			
PCIe	Xilinx Alveo U200 and Emulating PMEM 64 GB			
CXL-D				
CXL-B				
CXL				

TABLE 2. Device performance characteristics (normalized to DRAM performance).

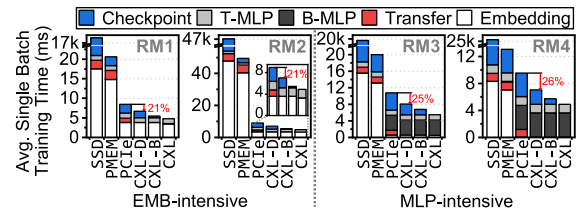
	Latency (versus DRAM)		Bandwidth (versus DRAM)	
	Read	Write	Read	Write
PMEM	3x	7x	0.6x	0.1x
SSD	165x		0.02x	

includes a CXL device (3.0) controller, four memory controllers, computing/checkpointing logic. The computing logic consists of a set of adders, multipliers, and scratchpad memory to store interim embedding vectors. The checkpointing logic has a CXL DMA engine and two counters, which deal with data copying for the embedding and MLP log. Note that the memory controllers emulate the PRAM latency by delaying DRAM responsiveness to make its memory performance similar to PMEM.¹¹ For CXL-GPU, we port Vortex¹² into our platform using Xilinx IPs (rather than its original Altera IPs). However, its instruction set architecture cannot support diverse CUDA kernels that our RM systems and workloads use. We thus emulate the kernel latency in Vortex by replaying per-batch MLP computation cycles, which are extracted from 3,090 GPU. The software interfaces, such as cuda-Memcpy, are implemented using Vortex's DMA engine working on a PCIe address space that our CXL and Xilinx IPs handle.

Test configurations: We prepare three different configurations, SSD (SSD), PMEM (PMEM), and PCIe-attached PMEM (PCIe) based on the underlying media where the embedding tables are stored into. While embedding operations of SSD and PMEM are performed on the host CPU, PCIe is capable of near-data

TABLE 3. Recommendation system models.

	RM1	RM2	RM3	RM4
Input dataset	Random	Random	Random	Criteo Kaggle
Features dim	32	32	32	16
# Dense features	13	13	13	13
# Embed. table	20	80	20	52
# Sparse features	80	80	20	1
Bottom-MLP	13-8192-2048-32	13-8192-2048-323	13-10240-4096-32	13-16384-2048-512-16
Top-MLP	256-64-1	512-128-1	512-128-1	512-128-1

**FIGURE 11.** Training time breakdown.

processing like our CXL-MEM. Moreover, SSD leverages host DRAM to cache embedding vectors. We also prepare three TRAININGCXL configurations to breakdown our contributions: 1) CXL-MEM without any scheduling supports (CXL-D); 2) CXL-D with batch-aware checkpoint (CXL-B); and 3) CXL-B with relaxed training techniques (CXL). Table 1 summarizes the detailed hardware specifications we used for all test configurations, while Table 2 lists up device performance characteristics, such as latency and bandwidth for read or write (which are normalized to that of DRAM).

Model configuration: We use open source DLRM benchmark and prepare four RMs for evaluation as listed in Table 3. RM1, RM2, and RM3 are based on model parameters from Gupta et al.'s work,¹³ while RM4 is based on Naumov et al.'s work.⁸ RM1 and RM2 are embedding-intensive models, requiring the lookup of 80 embedding vectors per embedding table. In particular, RM2 has 4× many embedding tables than RM1, making it the most embedding-intensive model among RM1~4. On the other hand, RM3 and RM4 are MLP-intensive models because the number of embedding vectors to lookup or the number of embedding tables is not that many compared to RM1 and RM2.

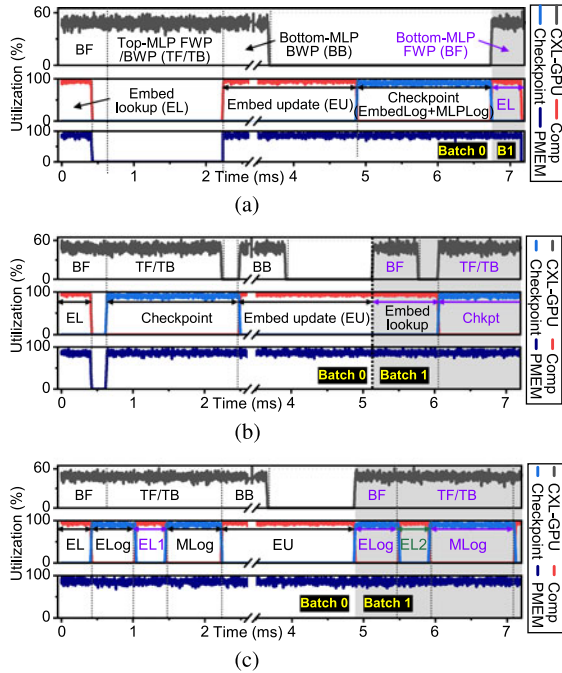


FIGURE 12. Utilization of hardware resources. (a) CXL-D. (b) CXL-B. (c) CXL.

Note that, we set the embedding table larger and the bottom-MLP deeper than the original model to reflect increases in dense and sparse features in the real world. In addition, we consider Criteo Kaggle's embedding table access distribution when randomly generating sparse feature input for RM1~3 to evaluate the RAW impact similar to the real datasets.

Overall Training Latency

Figure 11 shows RMs' average batch training time. For the embedding-intensive models (RM1/RM2), PMEM exhibits $949\times$ faster RM training time (including T-MLP, B-MLP, Transfer, and Embedding, except for Checkpoint) than SSD, on average. This is because PMEM can be accessed in a byte-addressable manner and its read/write is faster than SSD. Since embedding tables are stored across multiple PMEMs, PCIe, CXL-D, CXL-B, and CXL can parallelize the embedding vector accesses as well as increase the performance of embedding operations. Unfortunately, acceleration of embedding operations with NDP-capable PMEM does not work well for the MLP-intensive models compared to embedding-intensive models. This is because MLP-intensive models exhibit long latency of B-MLP than embedding-intensive models, thereby failing to fully overlap that latency with Embedding. Specifically, CXL-D shows 23% training time reduction compared to PCIe on average. Its performance benefits come from two reasons. First, all software

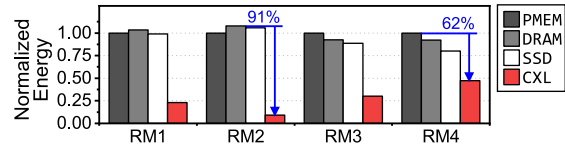


FIGURE 13. Energy analysis.

overheads caused by `cudaStreamSynchronize` and `cudaMemcpy` are eliminated by automatic data movement. Second, CXL-MEM's checkpointing logic can directly examine the MLP parameters updated by CXL-GPU (for each batch) through CXL.cache, which can hide the overhead behind the time to compute (embedding operations). CXL-B further improves training performance than CXL-D by overlapping the checkpointing with the CXL-GPU operations. Finally, our CXL can reduce training time by 14% compared to CXL-B by eliminating checkpointing and RAW overheads through relaxed training techniques.

Resource Utilization Analysis

We analyze the utilization of computing and memory resources, including CXL-GPU, CXL-MEM's computing logic, checkpointing logic, and PMEM. We breakdown utilization timelines with RM training operations to understand the proposed contributions, as shown in Figure 12.

Figure 12(a) shows training timeline of CXL-D. During 0.6~2.2 ms, CXL-MEM is idle while CXL-GPU performs feature interaction and top-MLP operations. After CXL-MEM finishes the embedding update operation at 4.9 ms, the checkpointing logic of CXL-MEM performs checkpointing in a redo log manner, and the next batch starts at 6.7 ms. Compared to CXL-D, CXL-B supports batch-aware checkpoints [cf., Figure 12(b)]. Therefore, when FWP of bottom-MLP is completed at 0.6 ms, CXL-MEM's checkpointing logic starts to perform checkpointing. Checkpointing overhead is observed during 2.2~2.5 ms since the checkpointing time is longer than the forward and BWP times of feature interaction and top-MLP operations. Nevertheless, the training time is reduced by 1.6 ms compared to CXL-D because checkpointing is performed by fully utilizing the idle time of CXL-MEM. Note that batch 1's embedding lookup time increases in CXL-B than CXL-D. This is because PMEM's RAW performance is observed due to consequent embedding update (at batch 0) and embedding lookup (at batch 1) operations.

With the proposed training relaxation techniques, CXL not only maximizes computing and memory resource utilization but also improves training time [cf., Figure 12(c)]. Relaxed batch-aware checkpoint is observed during 1.4~2.2 ms. Since MLP logging starts from 1.4 ms and stops when the CXL-GPU's top-MLP

operation is completed at 2.2 ms, the checkpointing overhead is eliminated. Relaxed embedding lookup is observed during 1~1.4 ms. By relaxing computational dependency to avoid RAW, embedding lookup time is shortened compared to CXL-B.

Power Analysis

In this section, we analyze energy consumption between SSD, PMEM, and CXL; the energy values of SSD and CXL are normalized to those of PMEM for better understanding. For comparison with high-performance training systems, we also add DRAM, which is an ideal configuration where the entire embedding tables are loaded into DRAM. As shown in Figure 13, our proposed CXL exhibits the lowest energy consumption across all RMs. However, the energy consumption difference between CXL and other configurations varies greatly between RMs, based on the type of computation intensity. For example, embedding-intensive RM2 shows the most significant energy savings, as much as 91% compared to DRAM, whereas MLP-intensive RM4 only shows as much as 62% compared to PMEM. The reason why embedding-intensive RMs show more energy savings than MLP-intensive RMs is mainly related to how much CXL-MEM shortens the training time.

Although DRAM can train RMs faster than SSD, it consumes more energy than SSD. This is because DRAM requires more memory modules to store the same size of embedding tables. That is why the energy consumption of DRAM is much higher than that of PMEM during embedding table accesses. However, when we compare the energy consumption of DRAM and PMEM, there are different trends between embedding-intensive models (RM1 and RM2) and MLP-intensive models (RM3 and RM4). In RM1 and RM2, DRAM consumes more energy than PMEM due to DRAM's large energy consumption than PMEM. On the other hand, the energy consumption of PMEM is higher than that of DRAM in RM3 and RM4. This is because PMEM should log the large amount of bottom and top-MLP's parameters, whereas DRAM does not perform checkpointing.

RELATED WORK

In-storage processing (ISP): Prior works proposed ISP to solve data movement overhead of SSD-based embedding table storage.^{14,15} While RecSSD¹⁴ considers embedding lookup as ISP, RM-SSD¹⁵ offloads the entire recommendation system to the underlying storage device to eliminate the significant GPU-SSD synchronization overhead further. In the sense that the CXL-MEM can alleviate data movement overhead, it is similar to the existing ISP works. However, TRAININGCXL is differentiated from existing works as it guarantees a data persistency, and all data movement is done by hardware without software intervention.

Near-memory processing: There are also DRAM-based near-memory processing schemes^{16,17} to achieve high-performance inference for recommendation systems. However, they suffer from the limitation of memory expansion. RecNMP¹⁶ is implemented with the existing memory interface (e.g., DDR4), which limits the memory module expansion with the number of CPUs. On the other hand, TensorDIMM¹⁷ proposes a disaggregated memory node that can pool TensorDIMMs by leveraging NVSwitch. However, there is a scalability limitation since NVSwitch cannot be employed in a multitiered manner. TRAININGCXL proposes a CXL-based memory disaggregation approach, which can easily be expanded with the CXL 3.0's multilevel switching support. In addition, thanks to the CXL-MEM's on-card computing capability, our work is differentiated from RecNMP and TensorDIMM, which exploits on-DIMM computing capability. CXL-MEM can perform near-data processing without modification of commodity memory modules, and its computing capability is not limited by the memory media capacity.

CONCLUSION

We propose TRAININGCXL that can efficiently process large-scale RMs in the disaggregated memory pool by integrating PMEM and GPU into a single cache-coherent domain. Our batch-aware checkpoint can effectively hide checkpointing overhead from the training process, and the relaxation of embedding lookup removes RAW conflict issues, thereby improving the training bandwidth. Overall, our evaluation demonstrates that TRAININGCXL achieves 5.2× training performance improvement while consuming 76% lower energy compared to the state-of-the-art PMEM-based recommendation systems.

OUR EVALUATION DEMONSTRATES
THAT TRAININGCXL ACHIEVES 5.2×
TRAINING PERFORMANCE
IMPROVEMENT WHILE CONSUMING
76% LOWER ENERGY COMPARED TO
THE STATE-OF-THE-ART PMEM-
BASED RECOMMENDATION SYSTEMS.

ACKNOWLEDGMENTS

We appreciate anonymous reviewers, and we would like to thank Panmnesia for all the technical support. This work is protected by one or more patents.

REFERENCES

1. K. Hazelwood et al., "Applied machine learning at Facebook: A datacenter infrastructure perspective," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2018, pp. 620–629.
2. D. Mudigere et al., "Software-hardware co-design for fast and scalable training of deep learning recommendation models," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, 2022, pp. 993–1011.
3. A. Eisenman et al., "Check-N-Run: A checkpointing system for training deep learning recommendation models," in *19th USENIX Symp. Networked Syst. Des. Implementation*, 2022, pp. 929–943.
4. Z. Wang et al., "Merlin hugeCTR: GPU-accelerated recommender system training and inference," in *Proc. 16th ACM Conf. Recommender Syst.*, 2022, pp. 534–537.
5. W. Zhao, J. Zhang, D. Xie, Y. Qian, R. Jia, and P. Li, "AlBox: CTR prediction model training on a single node," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 319–328.
6. C. Wu, C. Wu, G. Wei, and D. Brooks, "A joint management middleware to improve training performance of deep recommendation systems with SSDs," in *Proc. IEEE/ACM 59th Des. Autom. Conf.*, 2022, pp. 157–162.
7. CXL Consortium, "Compute Express Link 3.0 Specification," Aug. 2022. [Online]. Available: www.computeexpresslink.org
8. M. Naumov et al., "Deep learning recommendation model for personalization and recommendation systems," 2019, *arXiv:1906.00091*.
9. G. Park, M. Kwon, P. Mahapatra, M. Swift, and M. Jung, "BIBIM: A prototype multi-partition aware heterogeneous new memory," in *Proc. 10th USENIX Workshop Hot Top. Storage File Syst.*, 2018.
10. Y. Kwon and M. Rhu, "Training personalized recommendation systems from (GPU) scratch: Look forward not backwards," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, 2022, pp. 860–873.
11. Z. Wang, X. Liu, J. Yang, T. Michailidis, S. Swanson, and J. Zhao, "Characterizing and modeling non-volatile memory systems," in *Proc. IEEE/ACM 53rd Annu. Int. Symp. Microarchit.*, 2020, pp. 496–508.
12. B. Tine, K. Yalamarthy, F. Elsabbagh, and K. Hyesoon, "Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics," in *Proc. IEEE/ACM MICRO-54: 54th Annu. Int. Symp. Microarchit.*, 2021, pp. 754–766.
13. U. Gupta et al., "DeepRecSys: A system for optimizing end-to-end at-scale neural recommendation inference," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, 2020, pp. 982–995.
14. M. Wilkening et al., "RECSSD: Near data processing for solid state drive based recommendation inference," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2021, pp. 717–729.
15. X. Sun, H. Wan, Q. Li, C. Yang, T. W. Kuo, and C. Xue, "RM-SSD: In-storage computing for large-scale recommendation inference," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit.*, 2022, pp. 1056–1070.
16. L. Ke et al., "RecNMP: Accelerating personalized recommendation with near-memory processing," in *Proc. IEEE/ACM 47th Annu. Int. Symp. Comput. Archit.*, 2020, pp. 790–803.
17. Y. Kwon, Y. Lee, and M. Rhu, "TensorDIMM: A practical near-memory processing architecture for embeddings and tensor operations in deep learning," in *Proc. IEEE/ACM 52nd Annu. Int. Symp. Microarchit.*, 2019, pp. 740–753.

MIRYEONG KWON is currently working toward a Ph.D. degree at KAIST, Daejeon, 34141, South Korea. Her research interests include graph analysis, GPU/SSD integration, persistent memory, and CXL. Contact her at mkwon@camelab.org.

JUNHYEOK JANG is currently working toward a Ph.D. degree at KAIST, Daejeon, 34141, South Korea. His research interests include ML, NVMe, and CXL. Contact him at jhjang@camelab.org.

HANJIN CHOI is currently working toward an M.S. degree at KAIST, Daejeon, 34141 South Korea. His research interests include RISC-V and CXL. Contact him at hjchoi@camelab.org.

SANGWON LEE is currently working toward a Ph.D. degree at KAIST, Daejeon, 34141, South Korea, and also works at Panmnnesia, Inc. His research interests include persistent memory, RISC-V, and CXL. Contact him at swon@camelab.org.

MYOUNGSOO JUNG has been an associate professor in the School of Electrical Engineering, KAIST, Daejeon, 34141, South Korea, since 2019. He is also the CEO of Panmnnesia, inc. His research interests include computer architecture, operating system, storage systems, nonvolatile memory, parallel processing, heterogeneous computing, and CXL. He is the corresponding author of this article. Contact him at m.jung@kaist.ac.kr.